AI-POWERED

# Application Intelligente

## Détection des Risques de la Maladie d'Alzheimer

ANGULAR | SPRING BOOT | MICROSERVICES | TEMPS RÉEL

**Architecture Microservices**

Module Développement d'Applications

# Objectif du Projet

## Contexte

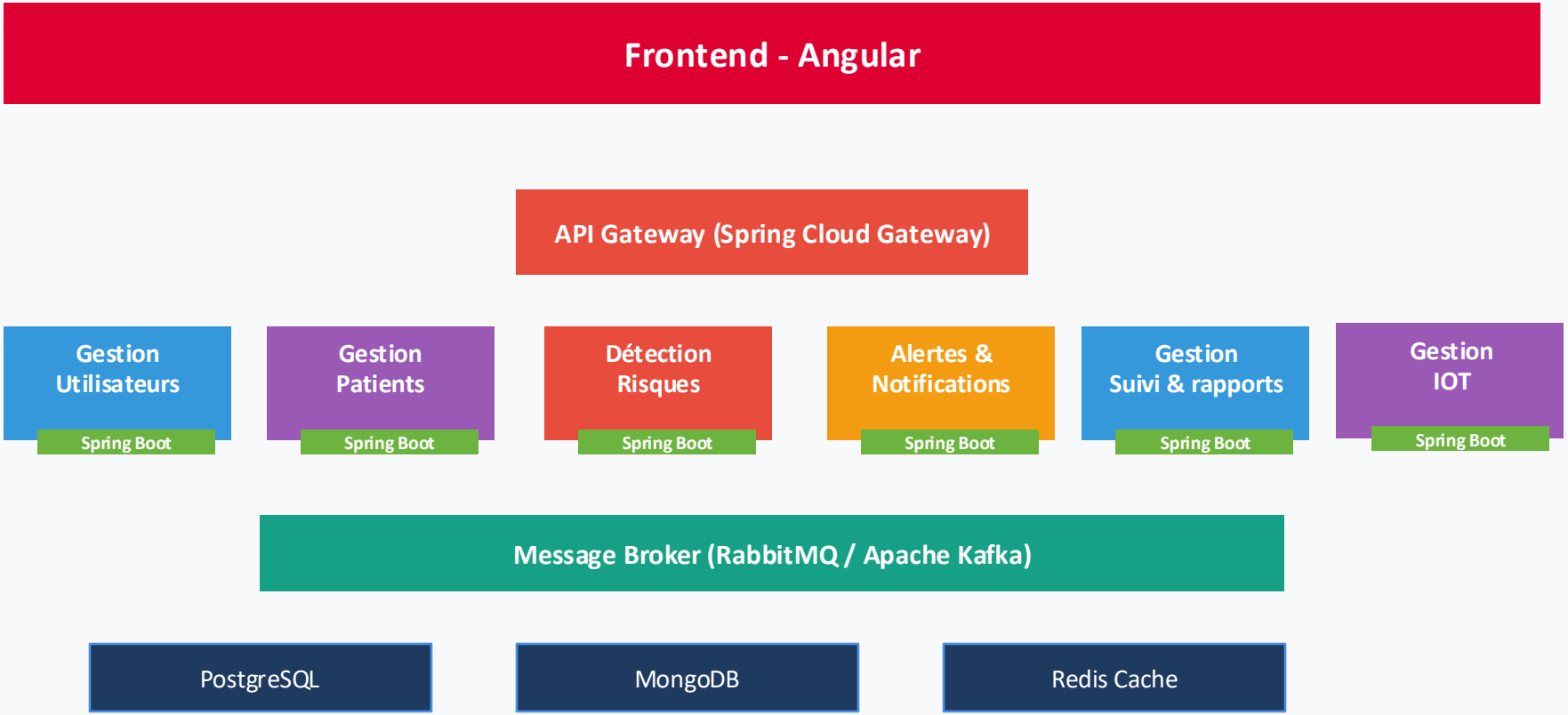
La prise en charge des patients atteints d'Alzheimer nécessite une surveillance continue pour prévenir les situations à risque (chutes, fugues, comportements anormaux). Les solutions actuelles manquent d'automatisation et de personnalisation.

## Solution Proposée

Développement d'une **application web intelligente** pour :

- • Gestion centralisée des patients Alzheimer
- • Détection automatique des situations à risque par règles métier
- • Génération d'alertes en temps réel
- • Suivi continu avec rôles différenciés (Admin, Aidant, Soignant, Patient)

# Architecture Globale Microservices



# Description des Microservices (1/2)

## Service Gestion Utilisateurs

**Responsabilité :** Authentification, autorisation, gestion des rôles (Admin, Aidant, Soignant, Patient)
**Technologies :** Spring Boot, Spring Security, JWT, PostgreSQL

## Service Gestion Patients

**Responsabilité :** CRUD patients, profils médicaux, historique, assignation aux aidants/soignants
**Technologies :** Spring Boot, Spring Data JPA, MongoDB

## Service Détection des Risques

**Responsabilité :** Moteur de règles métier, analyse comportementale, détection chutes/fugues/anomalies
**Technologies :** Spring Boot, Drools/ML, PostgreSQL

## Service Alertes & Notifications

**Responsabilité :** Génération et envoi d'alertes (email, SMS, push), gestion des priorités
**Technologies :** Spring Boot, Spring Mail, Twilio, Firebase

# Description des Microservices (2/2)

### Service Suivi & Rapports

**Responsabilité :** Tableaux de bord, statistiques, rapports d'évolution, exports PDF
**Technologies :** Spring Boot, Jasper Reports, PostgreSQL

### Service Données IoT (Optionnel)

**Responsabilité :** Collecte données capteurs (montres, bracelets), traitement temps réel
**Technologies :** Spring Boot, Kafka

## Récapitulatif Technologies

| Catégorie | Technologies |
|---|---|
| Frontend | Angular 17+, TypeScript, RxJS, Angular Material, NgRx |
| Backend | Spring Boot 3.x, Spring Cloud, Spring Security, Spring Data JPA |
| Bases de données | PostgreSQL, MongoDB, Redis, InfluxDB |
| Communication | REST API, WebSocket, RabbitMQ/Apache Kafka |
| Déploiement | Docker, Kubernetes, Spring Cloud Config |

# Communication & Flux de Données

### Communication Synchrone (REST API)

- • Angular ↔ Spring Cloud Gateway
- • API Gateway ↔ Microservices Spring Boot
- • Service Patients → Service Utilisateurs (vérification autorisations)

**Protocole :** HTTP/HTTPS avec JWT, Spring Security

### Communication Asynchrone (Event-Driven)

- • Détection Risques → Alertes (événement risque détecté)
- • IoT → Détection Risques (données capteurs)
- • WebSocket pour notifications temps réel Angular

**Protocole :** RabbitMQ/Kafka + Spring AMQP/Kafka

## Scénario : Détection d'une Chute

1. Capteur IoT détecte mouvement anormal → Service IoT (Spring Boot)
2. Service IoT publie événement 'mouvement_anormal' → RabbitMQ/Kafka
3. Service Détection Risques (Spring Boot) consomme → analyse avec règles métier Drools
4. Risque confirmé → Publication événement 'alerte_chute' sur Message Broker
5. Service Alertes (Spring Boot) consomme → envoi notifications via Spring Mail/Twilio
6. Notification temps réel Angular via WebSocket + Service Suivi enregistre l'incident

# Infrastructure & Déploiement

## 🐳 Conteneurisation Docker

- • Chaque microservice Spring Boot = conteneur Docker
- • Angular build → conteneur Nginx
- • docker-compose.yml pour orchestration locale
- • Kubernetes + Spring Cloud Kubernetes pour production

## 💾 Serveurs de Base de Données

- • PostgreSQL : données relationnelles (utilisateurs, patients)
- • MongoDB : données semi-structurées (logs, profils)
- • Redis : cache Spring et sessions
- • InfluxDB : séries temporelles IoT

## 🅰 Frontend Angular

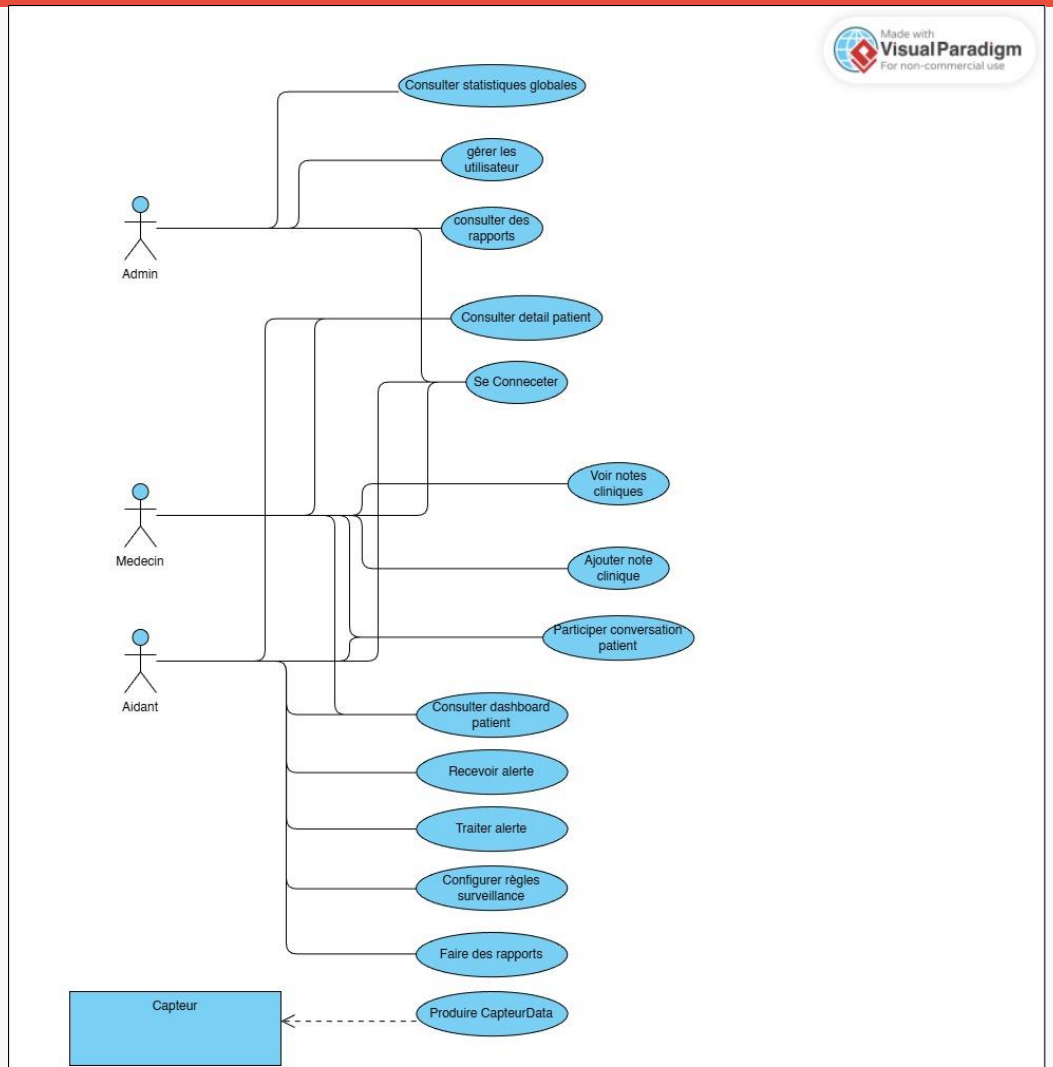
- • Framework : Angular 17+ avec TypeScript
- • Communication via HttpClient vers Spring Gateway
- • WebSockets (Socket.io) pour notifications temps réel
- • Build → Nginx dans conteneur Docker

## 🔒 Sécurité & Monitoring

- • Authentification : JWT avec Spring Security
- • HTTPS/TLS pour toutes communications
- • Logs : Spring Boot Actuator + ELK Stack
- • Monitoring : Spring Boot Admin + Prometheus + Grafana

## Pipeline CI/CD

*Git → GitLab CI/Jenkins → Maven/Gradle Build → Tests (JUnit, Mockito) → Build Docker → Push Registry → Deploy Kubernetes*


Made with
Visual Paradigm
For non-commercial use
Consulter statistiques globales
gérer les utilisateur
consulter des rapports
Admin
Consulter detail patient
Se Conneceter
Voir notes cliniques
Medecin
Ajouter note clinique
Participer conversation patient
Aidant
Consulter dashboard patient
Recevoir alerte
Traiter alerte
Configurer règles surveillance
Faire des rapports
Capteur
Produire CapteurData

# Synthèse des Choix Techniques

## 1. Architecture Microservices avec Spring Boot

**Services** Spring Boot indépendants,
**Scalable** avec Spring Cloud (Gateway, Config,

Angular

Angular ⟷ Spring Cloud Gateway / Spring Cloud Discovery

## 2. Frontend Angular Moderne

Angular 17+ avec TypeScript
- RxJS pour la réactivité
- Angular Material pour l'UI

## 3. Communication Hybride

**REST API** (Spring Web) + **WebSocket** + Message Broker (Spring AMQP/ Kafka)

HTTP Spring Web → WebSocket → Kafka

## 4. Conteneurisation & Orchestration

Docker pour tous les services +
**Kubernetes** avec Spring Cloud

docker kubernetes

## 5. Sécurité & Monitoring Spring Natif

Spring Security + JWT + Spring Boot Actuator + Prometheus + Grafana

Spring JWT + Grafana

*Stack technologique complète : Angular + Spring Boot pour une application robuste et scalable*

# Conclusion

Cette architecture microservices permet de construire une application intelligente, fiable et scalable, capable de surveiller en continu les patients Alzheimer, de détecter les risques en temps réel et d’alerter efficacement les acteurs concernés, tout en assurant une séparation claire des responsabilités et une facilité de maintenance